tikzgitgraph

Contents

1 Loading the package **1**

2 The gitgraph environment **1**

3 Declaring a repository: \repo **2**

4 Adding a commit: \commit **2**
4.1 Example: a single branch with three commits 2

5 Creating a branch: \branch **2**
5.1 Example: a branch 2

6 Merging branches: \merge **3**
6.1 Merge example: equal number of commits 3
6.2 Merge example: parent has more commits (parentlonger) 3
6.3 Merge example: child has more commits (childlonger) 4

1 Loading the package

```
\usepackage{tikzgitgraph}
```

The package automatically loads `tikz` and the `calc` library.

2 The gitgraph environment

All drawing commands must appear inside a `gitgraph` environment, which wraps a `tikzpicture`.

```
\begin{gitgraph}[options]
...
\end{gitgraph}
```

Options can also be set globally with `\gitgraphsetup{...}`.

Option	Default	Description
`dir`	`tb`	Flow direction: `tb` (top to bottom) or `bt` (bottom to top)
`step`	`0.75cm`	Vertical distance between consecutive commits
`xstep`	`1.6cm`	Horizontal distance per branch offset unit
`branchdir`	`right`	Side on which commit labels are placed (`right` or `left`)
`labelsep`	`0.2cm`	Gap between the commit dot and its label
`linewidth`	`1pt`	Stroke width of all lines
`dotradius`	`2pt`	Radius of the commit dot
`round`	`4pt`	Corner rounding of elbows

3 Declaring a repository: \repo

```
\repo[options]{branch-name}{coordinate}
```

Initialises a branch and places its starting cursor at `coordinate`. Must be called before any `\commit` on that branch. Branch names must not contain `-` or `:`.

Option	Default	Description
`color`	`black`	Line and dot colour for this branch
`main=true`	`false`	Draws this branch's lines on the top layer, above all other branches
`name`	(empty)	Text label shown at the starting coordinate

4 Adding a commit: \commit

```
\commit{branch-name}[options]{label}
```

Advances the branch cursor by one `step`, draws a connecting line from the previous commit, and places the dot and label.

Option	Default	Description
`id`	auto	Explicit node identifier, used when referencing this commit in `\branch` or `\merge`

4.1 Example: a single branch with three commits

```
\begin{gitgraph}
    \repo[main=true, color=blue, name=main]{main}{0,0}
    \commit{main}{commit1}
    \commit{main}{commit2}
    \commit{main}{commit3}
\end{gitgraph}
```

5 Creating a branch: \branch

```
\branch{source-branch:id}{new-branch}[options]
```

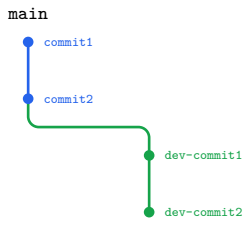
Creates a new branch forking from an existing commit. The source is written as `branch-name:commit-id`. A fork elbow is drawn automatically from the source commit to the position of the first future commit on the new branch.

Option	Default	Description
`color`	`black`	Colour of the new branch
`offset`	`1`	Horizontal position relative to the source branch, in multiples of `xstep`. Negative values place the branch to the left.

5.1 Example: a branch

```
\begin{gitgraph}
    \repo[main=true, color=blue, name=main]{main}{0,0}
        \commit{main}{commit1}
        \commit{main}{commit2}
```

```
        \branch{main:2}{dev}[color=green!60!black]
            \commit{dev}{dev-commit1}
            \commit{dev}{dev-commit2}
\end{gitgraph}
```



6 Merging branches: \merge

```
\merge{target-branch}[options]{source-branch:id}{label}
```

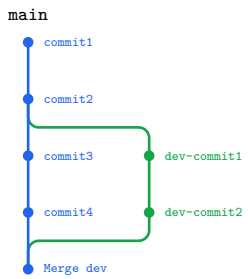
Adds a merge commit on `target-branch` and draws a merge line from the commit identified by `source-branch:id`.

Option	Default	Description
`id`	auto	Explicit id for the new merge commit
`parentlonger`	`false`	The source line drops vertically to align with the target. Use when the parent has more commits than the child.
`childlonger`	`false`	The target extends vertically to align with the source. Use when the child has more commits than the parent.

6.1 Merge example: equal number of commits

Parent and child branch both have two commits between the branch point and the merge. The default elbow routing works well.

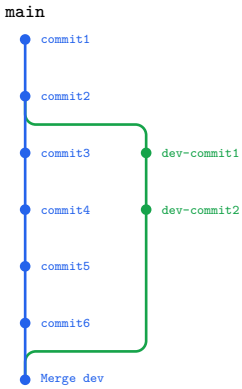
```
\begin{gitgraph}
    \repo[main=true, color=blue, name=main]{main}{0,0}
        \commit{main}{commit1}
        \commit{main}{commit2}
        \branch{main:2}{dev}[color=green!60!black]
            \commit{dev}{dev-commit1}
            \commit{dev}{dev-commit2}
        \commit{main}{commit3}
        \commit{main}{commit4}
        \merge{main}{dev:2}{Merge dev}
\end{gitgraph}
```



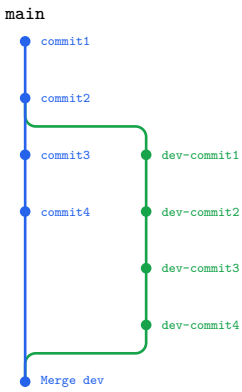
6.2 Merge example: parent has more commits (`parentlonger`)

The parent branch has four commits since the branch point, while the child only has two. Without `parentlonger` the merge line would be a steep diagonal; `parentlonger` makes the source line drop vertically first, then elbow horizontally into the target.

```latex
\begin{gitgraph}
    \repo[main=true, color=blue, name=main]{main}{0,0}
        \commit{main}{commit1}
        \commit{main}{commit2}
        \branch{main:2}{dev}[color=green!60!black]
            \commit{dev}{dev-commit1}
            \commit{dev}{dev-commit2}
        \commit{main}{commit3}
        \commit{main}{commit4}
        \commit{main}{commit5}
        \commit{main}{commit6}
        \merge{main}[parentlonger]{dev:2}{Merge dev}
\end{gitgraph}
```



6.3 Merge example: child has more commits (`childlonger`)

The child branch has four commits since the branch point, while the parent only has two. `childlonger` extends the target branch vertically to align with the source commit, then elbows the merge line in.

```latex
\begin{gitgraph}
    \repo[main=true, color=blue, name=main]{main}{0,0}
        \commit{main}{commit1}
        \commit{main}{commit2}
        \branch{main:2}{dev}[color=green!60!black]
            \commit{dev}{dev-commit1}
            \commit{dev}{dev-commit2}
            \commit{dev}{dev-commit3}
            \commit{dev}{dev-commit4}
        \commit{main}{commit3}
        \commit{main}{commit4}
        \merge{main}[childlonger]{dev:4}{Merge dev}
\end{gitgraph}
```